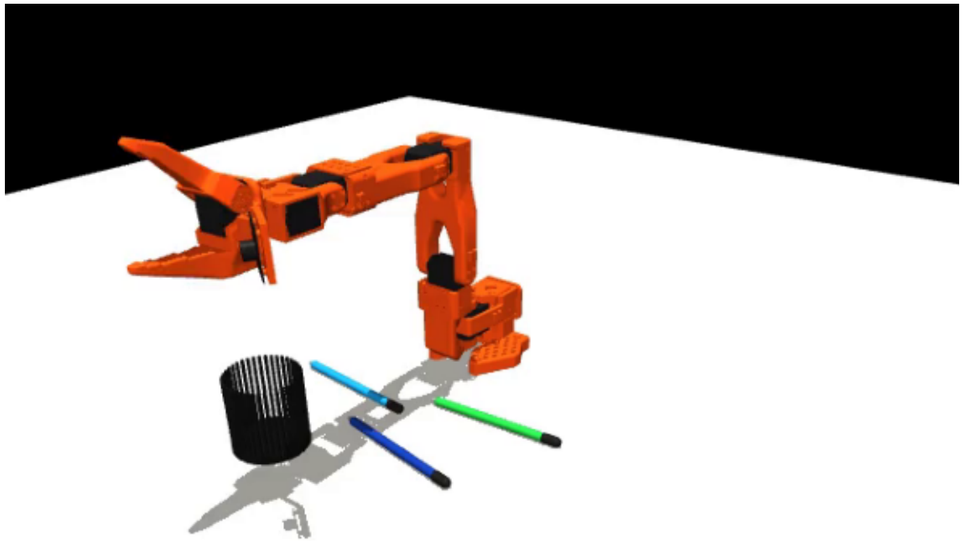
SO-101とπ0.5によるペン収納

机上のペンを、ロボットでペン立てに収納する。

このタスクでは、SO-101がペン3本を扱った。
タスクと教師データはCodexを用いて作成した。

動作の生成にはπ0.5を使い、上方・手首カメラの
画像と6関節の状態を入力した。



実行環境：SO-101、机上のペン3本、ペン立て

モデルへ与えた指示

Grab pens and place into pen holder

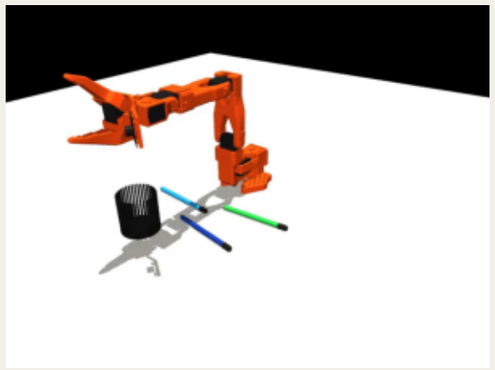
この指示で動かせるように追加学習し、
学習前後と、推論中も動かす場合の成否を確かめた。

ROS 2とMuJoCoを接続

MuJoCoから得た画像と関節状態を、ROS 2を通してπ0.5へ入力した。
π0.5が予測した位置指令は軌道コントローラで順に実行し、その結果を次の観測に使った。

MuJoCo 3.3.7

物理計算とカメラ画像の生成



画像と
関節状態

モデルへの入力を整える

message_filtersで観測の時刻をそろえ
関節の順序・単位と画像形式を変換

入力

π0.5で指令を予測

2視点の画像＋6関節の状態を入力
50指令のアクションチャンクを生成

50指令

ROS 2の軌道コントローラで実行

joint_trajectory_controller
50指令すべてを30 Hz（約1.67秒）で順に実行

関節の
位置指令

物理計算と軌道制御には公開パッケージを使い、観測の変換・モデル接続・起動設定をタスク用に実装した。

URDF：TheRobotStudio/SO-ARM100。Onshape CADからonshape-to-robotで出力されたモデルを使用。

物理形状：MuJoCo Menagerieのrobotstudio_so101。URDFに制御設定を加え、MJCFにペン・ペン立てを配置。

ROS 2 JazzyとGPU推論をDocker内に構成。設定：Dockerfile、compose.yaml、config/controllers.yaml、config/cameras.yaml

SO-101用のπ0.5を、ペン収納に追加学習

ベースモデル：dongyoonkim/so101-pi05-base

SO-101用のπ0.5を複数試し、その中で追加学習前でも
動作が滑らかだったこの重みを選んだ。

次に、物体位置と逆運動学から収納の成功デモを作成し、
55試行・99,000フレームでLoRAによる模倣学習を行った。
物体位置はデモの生成にだけ使い、モデルへは入力しなかった。

ベースの重みを固定し、LoRAの重みだけを更新

- LoRA：rank 16、alpha 16、dropout 0
- バッチサイズ：8、学習ステップ数：30,000
- 最適化：AdamW、最大学習率：0.00005
- 学習率：1,000ステップのウォームアップ後、cosineで減衰

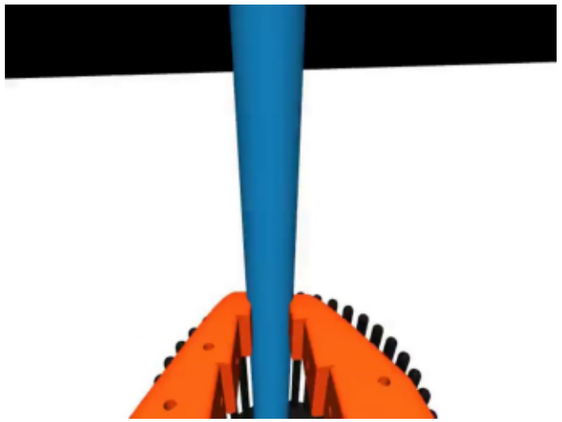
学習時は、訓練用と検証用の両方に55試行すべてが読み込まれた。
そのため、独立した検証データによる学習時評価は得られていない。

推論・学習の実装：SO-101対応のVLASH／LeRobot／PEFT

Codexで作成した教師データ



上方カメラ



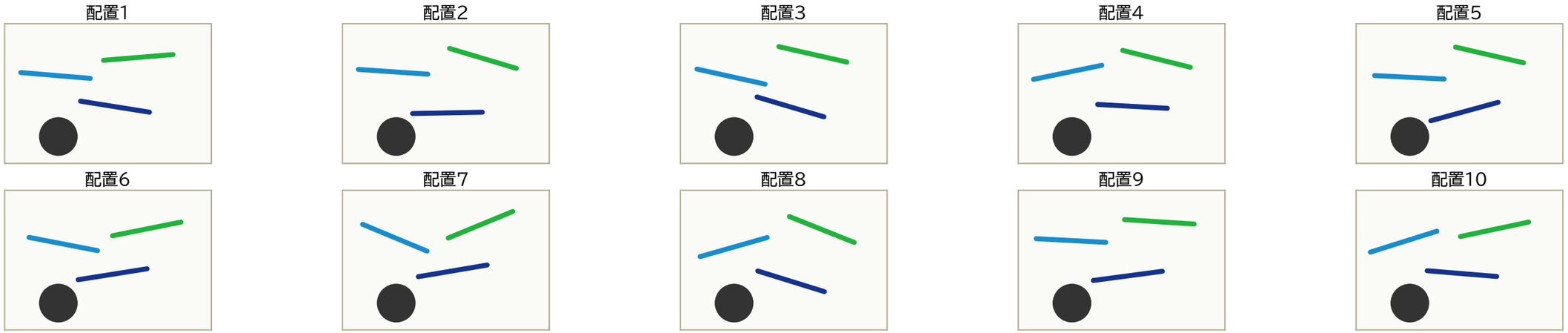
手首カメラ

同じ10配置で評価

追加学習の効果を確かめるため、ペンの位置と向きを変えた10配置を用意した。
各評価では同じ配置と推論時の乱数を使い、入出力の正規化も同じタスク用統計にそろえた。

各試行ではシミュレータ内の時間を75秒進め、3本すべてを収納し、
グリッパーから離れた状態を2秒維持できた場合を成功とした。

使用した10配置を、真上から見た模式図

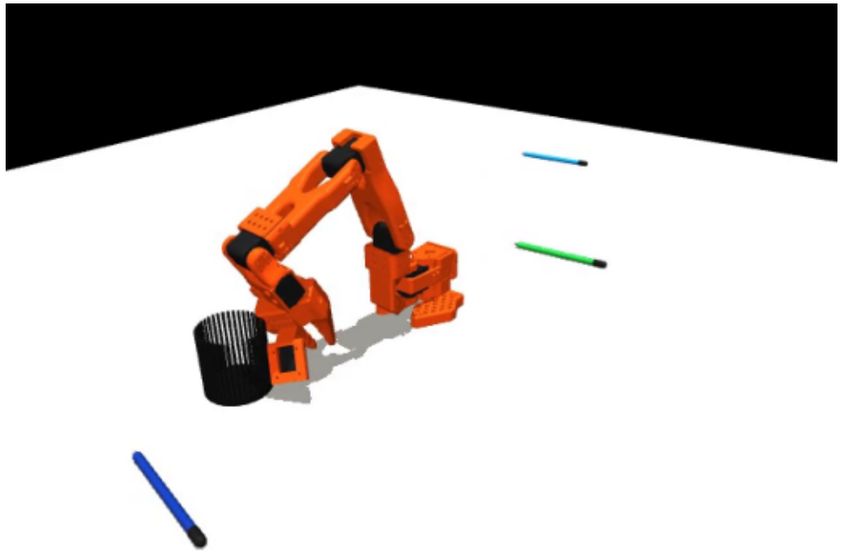


判定には記録した関節・ペンの状態を使用。模式図の線はペン、黒い円はペン立ての初期位置。

追加学習後に、収納の成功例を確認

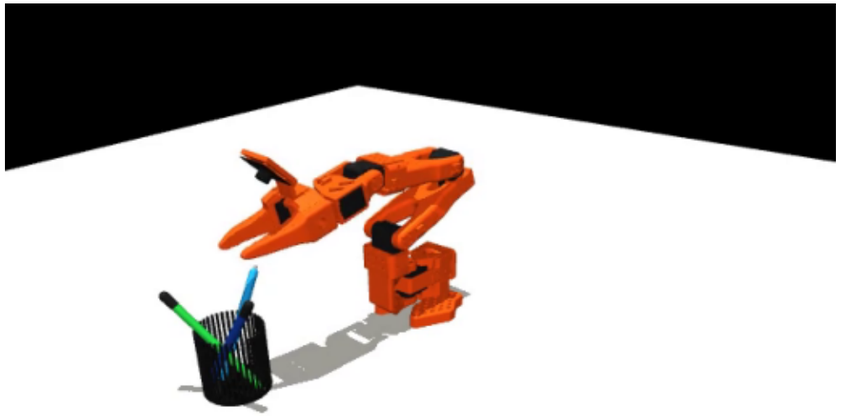
この10配置では、ベースモデルは10回中0回、追加学習後は10回中2回成功した。
ここでは両方とも推論中にシミュレーションを止め、計算待ちの間は物体が動かない条件にそろえた。

ベースモデル｜10回中0回成功



同じ初期配置から75秒間実行した後の状態：収納できず

追加学習後｜10回中2回成功



同じ初期配置から75秒間実行した後の状態：3本を収納・開放

右は成功した1試行の例。評価全体では、追加学習後も8回は成功基準に届かなかった。

推論中もシミュレーションを継続

前ページの評価では、推論中のシミュレーションを停止した。そこで実機を見据え、同じ重み・10配置で、
推論中も物体が動く条件で試した。その結果、成功は10回中1回で、ぎくしゃくした動きも見られた。

前の動作を実行しながら、次の指令を予測

観測・推論を開始

前のチャンク終了の約0.35秒前

→

前のチャンクを完了

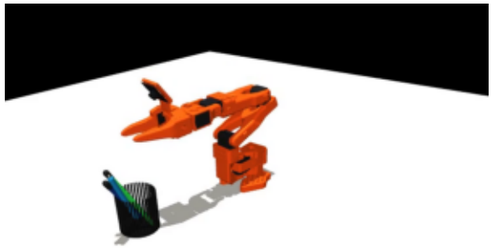
全50指令の実行が終了

→

次の指令を送信

先読みした予測を実行

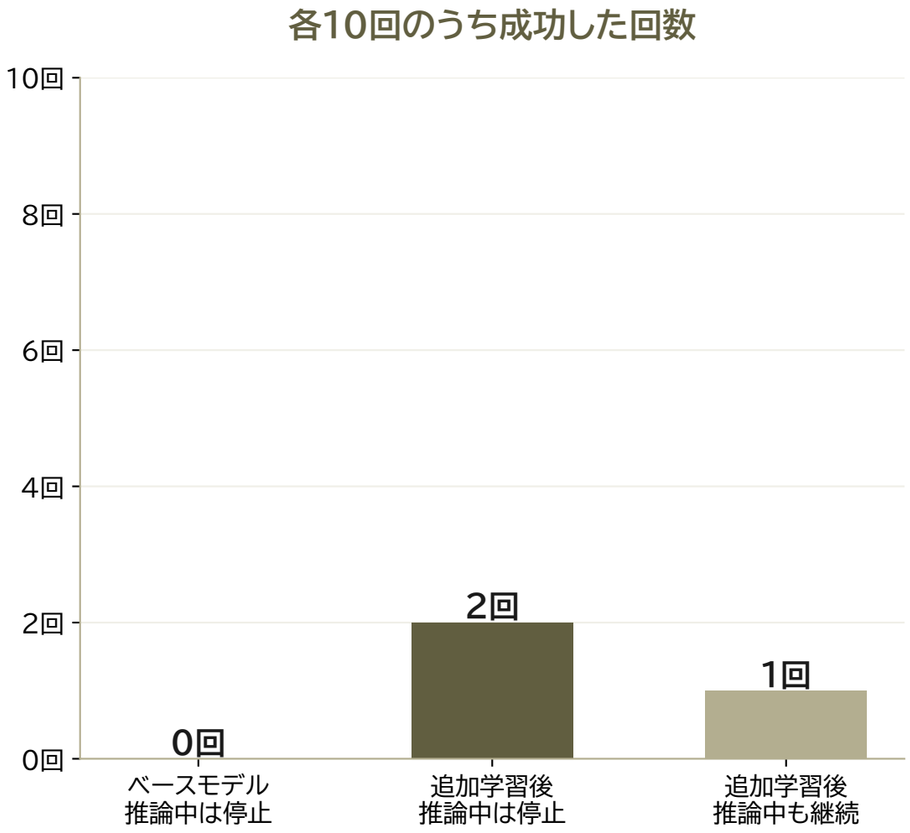
推論時間の中央値は0.14秒で、計算は動作中に進められた。
ただし、前のチャンクの終了を待ってから送ったため、
送信時の観測は中央値0.38秒前のものだった。



継続条件で成功した試行の終了時

0.38秒は推論時間ではなく、観測から送信までの経過時間。
この時間差には、前のチャンクが終わるまでの待ち時間も含まれる。

まとめ



追加学習で、ベースモデルにはなかった成功例を得た。

一方、同じ重みと配置でも、推論中に動かし続けると
成功した配置が変わり、ぎくしゃくした動きも残った。

先読みで推論待ちはほぼなくせたが、送信時の観測は
中央値0.38秒前のものだった。推論を先回りさせた分、
実行時点の状態とのずれにも対処すべきだった。

**次はこの時間差を踏まえて制御を見直し、
実機でも安定して動かせるように取り組みたい。**

結果は今回のタスク・10配置における評価。実機での動作は未評価。